HK-2 Human Robot Teaming

CS 4850 – Section 01 – Fall 2025

October 26th, 2025

Sharon Perry

Britney Lam
Documentation
Developer

Jalon Jones
Team Leader
Documentation
Developer

Name	Role	Cell Phone	Alternate Email
Britney	Project Owner Developer and Documentation	808.699.1350	blam6@students.kenneaw.edu
Jalon	Project Advisor Developer and Documentation	470.214.0574	jjon906@students.kennesaw.edu

Development Overview

Basic Overview

Our project integrates a Large Language Model (LLM) into Webots, a robotic simulation environment, to create a collaboration between human given tasks and robots. The system allows for a simulated robot in the program to interpret natural-language commands, generate structured action plans, and execute these actions in the simulated world. Our implementation intends to blend natural-language understanding and autonomous planning.

Start of Development

Development started by setting up Webots in a Linux-based environment, specifically Ubuntu and integrating the Microsoft Phi-3 Mini 4K Instruct model through Python. The LLM would take user input and produce a JSON-formatted action plan that the robot can read. Each JSON object represents ordered steps that correspond to simulated motions like movement, or high-level objectives like sensors that detect items in the simulated world. So far the beginning of our work has revolved around making sure that our translation pipeline can be structured into proper JSON output.

Python Scripts and Testing

The planner module was designed in Python to communicate directly with the LLM using Transformers and Accelerate libraries. There were initially a few problems with dependency management no the KSU HPC cluster, including some mismatches for PyTorch and libzstd. However, after troubleshooting, we were able to build a custom Conda environment which contained all of the required packages. We then created and tested a Python script that was made to verify the accuracy and consistency of the generated plans by running dry tests and logging the LLM's JSON responses. Another Python file, "metrics" aggregates and visualizes these results which computes the model's performance across multiple test cases. The metrics included the number of correctly executed actions and the similarities between the intended behavior of the simulations. The framework was most useful for comparing results between the different LLMs we had used, like Phi-3 Mini and Qwen 2.5 Instruct, without the need of drastically altering code each time between testing.

Database Connection

Not Applicable.

Everything is stored in a directory, so there's no use for SQL in the project. Webots executes locally, and the planner uses a JSON that is saved in an environment which can be accessed whenever necessary.

Project Setup and Execution

First Setup

Setting up our environment involves configuring the Python planner environment and the Webots simulation. First however, we need to SSH to the Kennesaw HPC. The information to do so is on their website. Once connected into the HPC, typing "module avail" will list all of the available modules that the HPC has, Anaconda is the only module that we need, so we need to download the latest version of it. "module load [exact module_name].

Second Setup

Next, create a regular conda environment. Once inside, you can download libraries using "conda forge [command]". However, some libraries aren't available to conda, so it's better to use pip to install.

Third Setup/Troubleshooting

Now, we must make sure that we have all of the dependencies to work. Torch, Transformers, and Safetensors. Using pip for these is recommended as Conda may have issues with these dependencies—as aforementioned — which can be found with "pip install –r.requirements.txt" or manually. Finding these dependencies through requirements.txt can be google searched and downloaded into the environment.

Final Setup

Now, we can download our Python scripts and keep them in the same environment. Planner.py, Executor.py, Metrics.py, and Schema.py. Once done, we can then generate a plan using our LLM. Once we have a plan generated, the schema exposes the JSON and makes sure that everything is correct. Then the executor will create a mock robot that will log the actions and give navigational coordinates, manipulating objects, or communicating etc. Then the metrics will score the plan and rate the accuracy similarity or the length and give an overall percent rating.